If you think you're better off without handling exception Z, you're right. `SupervisorStrategy.stop` is by default the strategy that handles every nonfatal exception.

For readability, you can create a single function to use with your behavior with all these strategies (backoff included):

```
package faulttolerance3

object DbWriter {
  ...

  def supervisorStrategy(behavior: Behavior[Command]): Behavior[Command] =
    Behaviors
      .supervise {
        Behaviors
          .supervise(behavior)
          .onFailure[UnexpectedColumnsException]
          .SupervisorStrategy.resume
      }
      .onFailure[DbBrokenConnectionException](SupervisorStrategy
        .restartWithBackoff(minBackoff = 3, maxBackoff = 30, randomFactor =     0.1)
        .withResetBackoffAfter(15))

}
```

You use the function like this: `supervisorStrategy(someBehavior)`.

5.4 Watching signals from an actor

An actor can watch another actor and be notified when the observed actor stops. The signals a watching actor can receive are `Terminated`, its subclass `ChildFailed`, or a custom class. A custom class is equivalent to `Terminated` but contains more information that may be relevant to the listener. `Terminated` or